

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

August 28, 2025

Lecture 4

Machine Learning: Overview

Human Learning

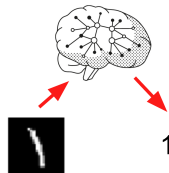
Observation



Learning



Decision rule



Machine Learning

Training Data



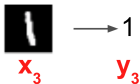
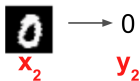
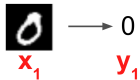
Machine Learning



Decision rule

Machine Learning

Training Data



Machine Learning



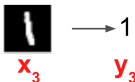
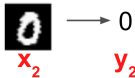
x_1 : vector of pixel values [0, 24, 128, ...]

y_1 : 0 or 1

Decision rule

Machine Learning

Training Data



Machine Learning



Decision rule

g (a function)

x

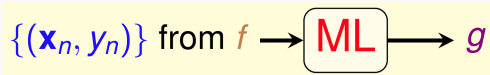


0 or 1

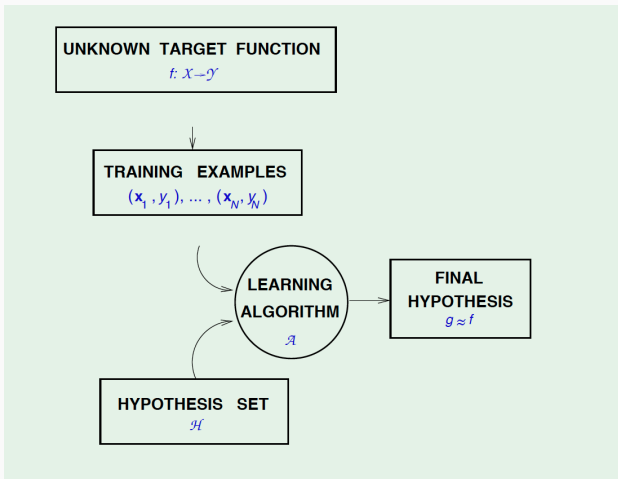
g maps any image (vector) to 0/1

Formalize the Learning Problem

- Input: $\mathbf{x} \in \mathcal{X}$ (an image)
- Output: $y \in \mathcal{Y}$ (class)
- Target function to be learned:
 $f : \mathcal{X} \rightarrow \mathcal{Y}$ (ideal image classification function)
- Data:
 $\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$
- Hypothesis (model)
 $g : \mathcal{X} \rightarrow \mathcal{Y}$ (**learned** formula to be used)



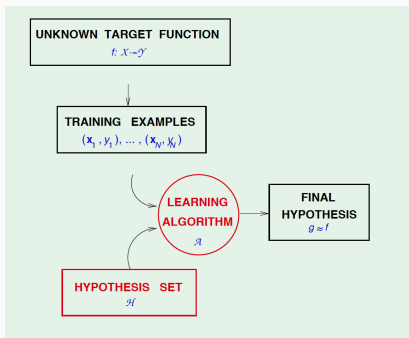
Basic Setup of Learning Problem



(Figure from “Learning from Data”)

Learning Model

- A learning model has two components:
 - The **hypothesis set** $\mathcal{H}$:
Set of candidate hypothesis (functions)
 - The **learning algorithm**:
To pick a hypothesis (function) from the $\mathcal{H}$
Usually **optimization algorithm** (choose the best function to minimize the **training error**)



Other types of output space - Regression

- Regression: $y_n \in$ (output is a real number)
- Example:
 - Stock price prediction
 - Movie rating prediction
 - ...

Other types of output space - Multi-class prediction

Multi-class classification:

- $y_n \in \{1, \dots, C\}$ (C -way classification)
- Examples: hand-written digit recognition, object classification, ...



MNIST

airplane

automobile

bird

cat

deer

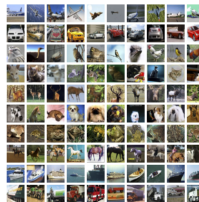
dog

frog

horse

ship

truck



CIFAR

Other types of output space - Multi-label prediction

- Multi-class problem: Each sample only has **one label**
- Multi-label problem: Each sample can have **multiple labels**

Other types of output space - Multi-label prediction

- Multi-class problem: Each sample only has **one label**
- Multi-label problem: Each sample can have **multiple labels**
- Example:
 - Document categorization (news/sports/economy/...)
 - Document/image tagging
 - ...

Other types of output space - Multi-label prediction

- Multi-class problem: Each sample only has **one label**
- Multi-label problem: Each sample can have **multiple labels**
- Example:
 - Document categorization (news/sports/economy/...)
 - Document/image tagging
 - ...
- **Extreme classification** (large output space problems):
 - Millions of billions of labels (but usually each sample only has few labels)
 - Recommendation systems: Predict a subset of preferred items for each user
 - Document retrieval or search: Predict a subset of related articles for a query

Even simpler Markov assumption: Unigram model

$$P(w_1 w_2 \dots w_n) \approx \prod_i P(w_i)$$

Some automatically generated sentences from Shakespeare unigram models

To him swallowed confess hear both . Which .
Of save on trail for are ay device and rote
life have

Hill he late speaks ; or ! a more to leg less
first you enter

Bigram model

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-1})$$

Some automatically generated sentences from Shakespeare bigram models

Why dost stand forth thy canopy, forsooth; he is
this palpable hit the King Henry. Live king.
Follow.

What means, sir. I confess she? then all sorts, he
is trim, captain.

More complex Markov assumption: Trigram model

$$P(w_i | w_1 w_2 \dots w_{i-1}) \approx P(w_i | w_{i-2} w_{i-1})$$

Some automatically generated sentences from Shakespeare trigram models

Fly, and will rid me these news of price.
Therefore the sadness of parting, as they say,
'tis done.

This shall forbid it should be branded, if renown
made it empty

Estimating bigram probabilities

The Maximum Likelihood Estimate

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

An example

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

Raw bigram counts

Out of 9222 sentences

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Raw bigram probabilities

Normalize by unigrams:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

Result:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Bigram estimates of sentence probabilities

$P(< s> \text{ I want english food } < /s>) =$

$P(\text{I} | < s>)$

$\times P(\text{want} | \text{I})$

$\times P(\text{english} | \text{want})$

$\times P(\text{food} | \text{english})$

$\times P(< /s> | \text{food})$

$= .000031$

Dealing with scale in large n-grams

LM probabilities are stored and computed in log format, i.e. **log probabilities**

This avoids underflow from multiplying many small numbers

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

If we need probabilities we can do one exp at the end

$$p_1 \times p_2 \times p_3 \times p_4 = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

How to evaluate N-gram models

"Extrinsic (in-vivo) Evaluation"

To compare models A and B

1. Put each model in a real task
 - Machine Translation, speech recognition, etc.
2. Run the task, get a score for A and for B
 - How many words translated correctly
 - How many words transcribed correctly
3. Compare accuracy for A and B

Intrinsic (in-vitro) evaluation

Extrinsic evaluation not always possible

- Expensive, time-consuming
- Doesn't always generalize to other applications

Intrinsic evaluation: **perplexity**

- Directly measures language model performance at predicting words.
- Doesn't necessarily correspond with real application performance
- But gives us a single general metric for language models
- Useful for large language models (LLMs) as well as n-grams

Choosing training and test sets

- If we're building an LM for a specific task
 - The test set should reflect the task language we want to use the model for
- If we're building a general-purpose model
 - We'll need lots of different kinds of training data
 - We don't want the training set or the test set to be just from one domain or author or language.

Training on the test set

We can't allow test sentences into the training set

- Or else the LM will assign that sentence an artificially high probability when we see it in the test set
- And hence assign the whole test set a falsely high probability.
- Making the LM look better than it really is

This is called “Training on the test set”

Bad science!

Intuition of perplexity as evaluation metric:
How good is our language model?

Intuition: A good LM prefers "real" sentences

- Assign higher probability to “real” or “frequently observed” sentences
- Assigns lower probability to “word salad” or “rarely observed” sentences?

Intuition of perplexity 4: Use perplexity instead of raw probability

- Probability depends on size of test set
 - Probability gets smaller the longer the text
 - Better: a metric that is **per-word**, normalized by length
- **Perplexity** is the inverse probability of the test set, normalized by the number of words

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}}\end{aligned}$$

Perplexity

1. $S = (w_1, w_2, \dots, w_m)$
2. $\text{Perplexity}(S) = P(w_1, w_2, \dots, w_m)^{\frac{-1}{m}}$
3. $P(w_1, w_2, \dots, w_m) = P(w_1)P(w_2|w_1)P(w_m|w_{m-1} \dots w_1)$
4. $\log(\text{Perplexity}(S)) = \frac{1}{m} \sum -\log(P(w_m|w_{m-1} \dots w_1))$
5. Intuitively, it's weighted number of possible choices for the next word.

Intuition of perplexity 7: Weighted average branching factor

Perplexity is also the **weighted average branching factor** of a language.

Branching factor: number of possible next words that can follow any word

Example: Deterministic language $L = \{\text{red, blue, green}\}$

Branching factor = 3 (any word can be followed by red, blue, green)

Now assume LM A where each word follows any other word with equal probability $\frac{1}{3}$

Given a test set $T = \text{"red red red red blue"}$

$$\text{Perplexity}_A(T) = P_A(\text{red red red red blue})^{-1/5} = ((\frac{1}{3})^5)^{-1/5} = (\frac{1}{3})^{-1} = 3$$

But now suppose red was very likely in training set, such that for LM B:

- $P(\text{red}) = .8 \quad p(\text{green}) = .1 \quad p(\text{blue}) = .1$

We would expect the probability to be higher, and hence the perplexity to be smaller:

$$\begin{aligned} \text{Perplexity}_B(T) &= P_B(\text{red red red red blue})^{-1/5} \\ &= (.8 * .8 * .8 * .8 * .1)^{-1/5} = .04096^{-1/5} = .527^{-1} = 1.89 \end{aligned}$$

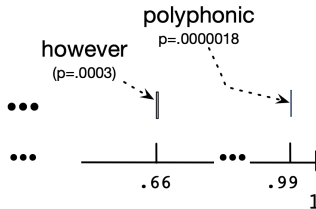
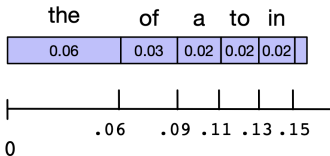
Perplexity

Holding test set constant:
Lower perplexity = better language model

Training 38 million words, test 1.5 million words, WSJ

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

Sampling a word from a distribution



The problem of sparsity

Maximum likelihood estimation has a problem

- Sparsity! Our finite training corpus won't have some perfectly fine sequences
 - Perhaps it has "ruby" and "slippers"
 - But happens not to have "ruby slippers"

This sparsity can take many forms

Verbs have direct objects; those can be sparse too!

Count(w | denied the)

Counts:

3 allegations

2 reports

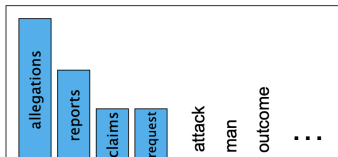
1 claims

1 request

0 attack

0 outcome

7 total



The intuition of smoothing

(Example modified from Dan Klein!)

When we have sparse statistics:

Count(w | denied the)

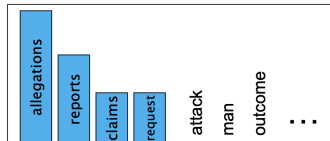
3 allegations

2 reports

1 claims

1 request

7 total



Steal probability mass to generalize better

Count(w | denied the)

2.5 allegations

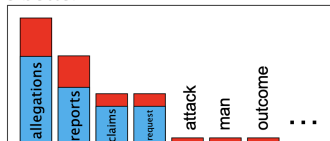
1.5 reports

0.5 claims

0.5 request

2 other

7 total



Add-one estimation

- Also called **Laplace smoothing**
- Pretend we saw each word one more time than we did
- Just add one to all the counts!

MLE estimate:

$$P_{\text{MLE}}(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

Add-1 estimate:

$$P_{\text{Laplace}}(w_n | w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{\sum_w (C(w_{n-1}w) + 1)} = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

Backoff and Interpolation

Sometimes it helps to use a simpler model

- Condition on **less** context for contexts you know **less** about

Backoff:

- If enough evidence, use trigram $P(w_n | w_{n-2}w_{n-1})$
- If not, use bigram $P(w_n | w_{n-1})$
- Else unigram $P(w_n)$

Interpolation:

- mix unigram, bigram, trigram

Interpolation works better

Linear Interpolation

Simple interpolation

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) = & \lambda_1 P(w_n|w_{n-2}w_{n-1}) \\ & + \lambda_2 P(w_n|w_{n-1}) \\ & + \lambda_3 P(w_n)\end{aligned} \quad \sum_i \lambda_i = 1$$

Lambdas conditional on context:

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) = & \lambda_1(w_{n-2}^{n-1}) P(w_n|w_{n-2}w_{n-1}) \\ & + \lambda_2(w_{n-2}^{n-1}) P(w_n|w_{n-1}) \\ & + \lambda_3(w_{n-2}^{n-1}) P(w_n)\end{aligned}$$

How to set λ s for interpolation?

Use a **held-out** corpus

Training Data

Held-Out
Data

Test
Data

Choose λ s to maximize probability of held-out data:

- Fix the N-gram probabilities (on the training data)
- Then search for λ s that give largest probability to held-out set

Backoff

Suppose you want:

$P(\text{pancakes} \mid \text{delicious soufflé})$

If the trigram probability is 0, use the bigram

$P(\text{pancakes} \mid \text{soufflé})$

If the bigram probability is 0, use the unigram

$P(\text{pancakes})$